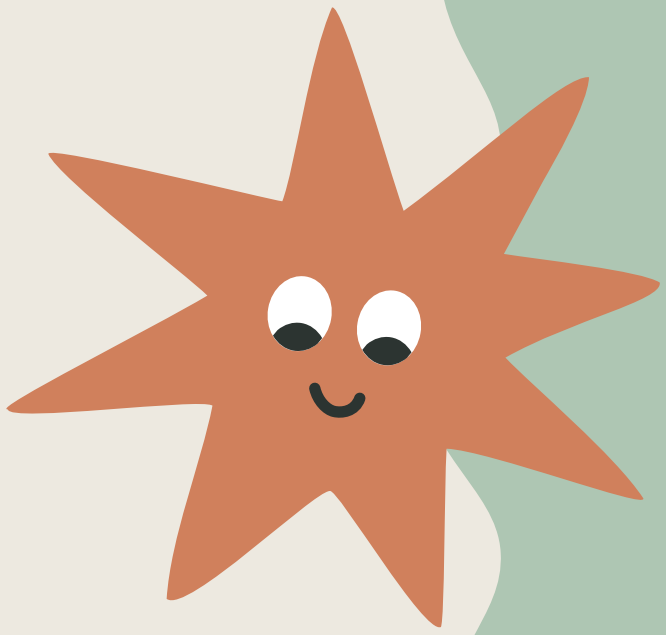
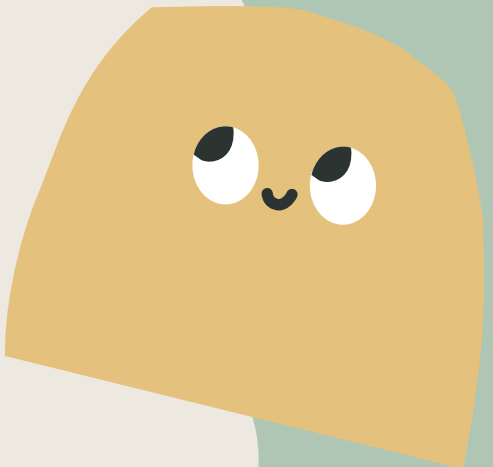
# Depth-First Search (DFS) in Artificial Intelligence

Lecture 12





# Learning Objectives

➢By the end of this lecture, students should be able to:

- Define Depth-First Search and explain why it is an uninformed search algorithm.
- Explain how DFS explores one branch deeply before backtracking.
- Understand the role of the stack and the LIFO principle.
- Trace DFS manually on a graph or search tree.
- Distinguish DFS traversal order from the solution path.
- Explain DFS completeness, optimality, and time and space complexity.
- Compare BFS and DFS and identify suitable applications for each.

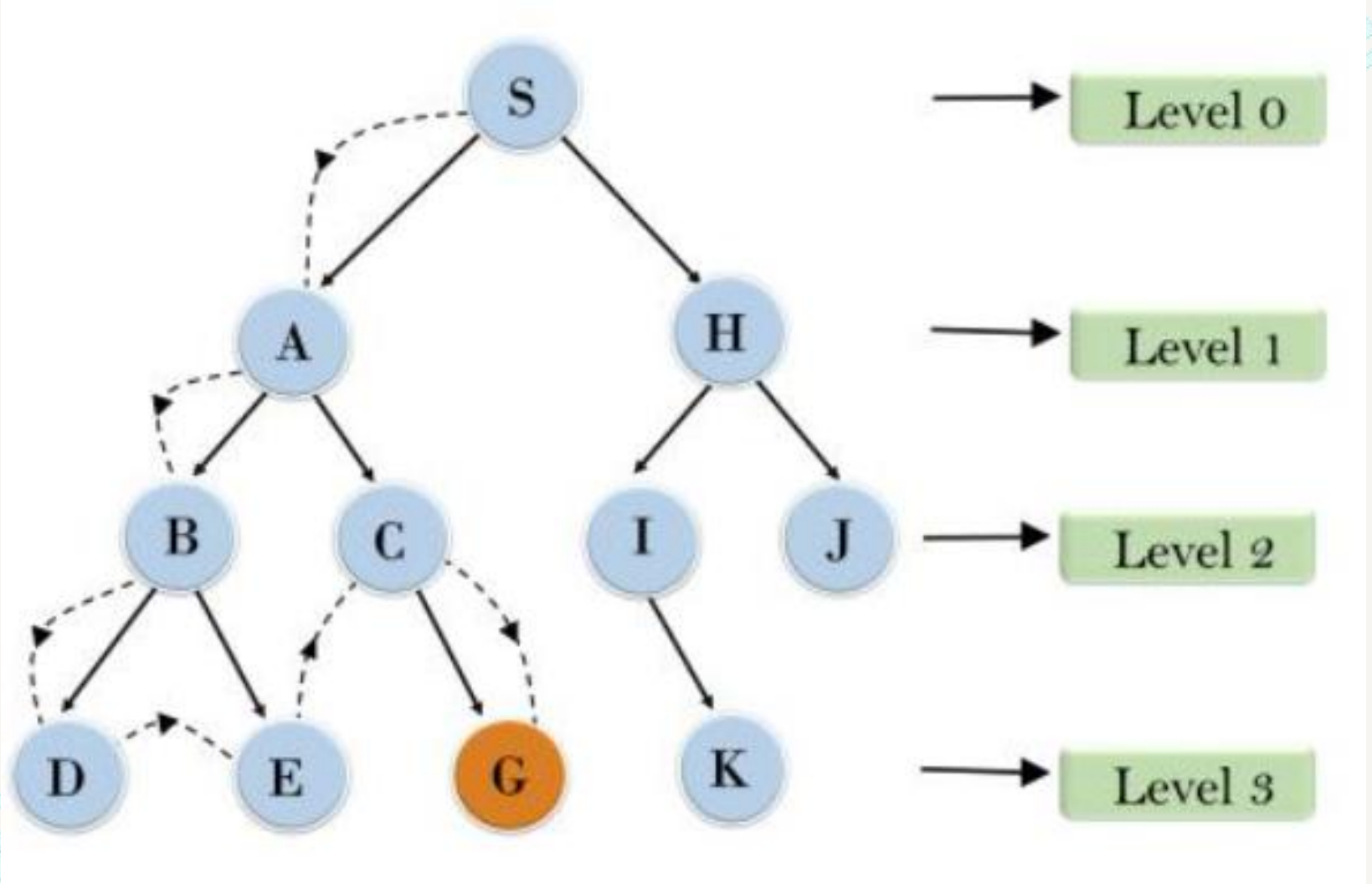
Prepared By: Ehtisham Rasheed

# What Is Depth-First Search?

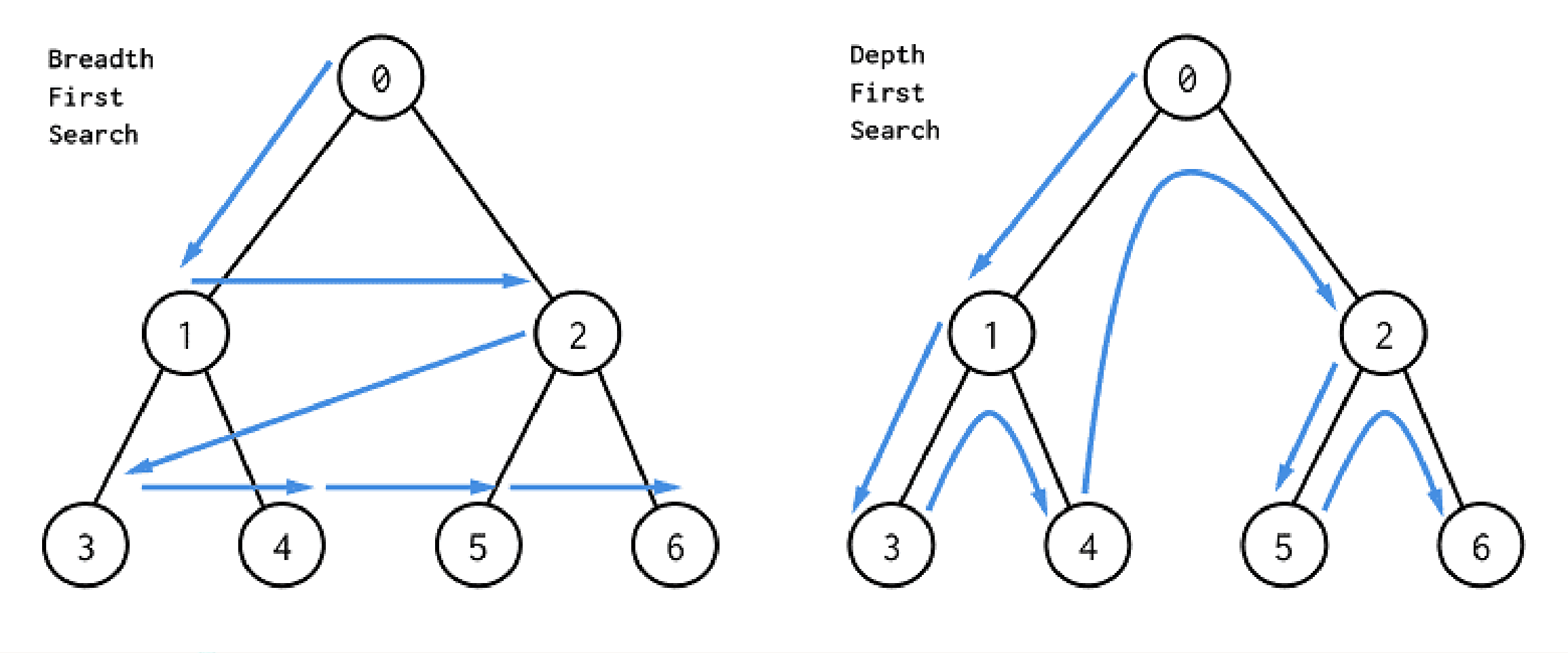
- Depth-First Search (DFS) is an uninformed search algorithm that explores a path as deeply as possible before backtracking to explore alternative paths
- DFS is called uninformed because it does not use a heuristic to estimate which node is closer to the goal
- It follows a depth-first exploration strategy

Prepared By: Ehtisham Rasheed

# What Is Depth-First Search?



# Understanding DFS Through an Example



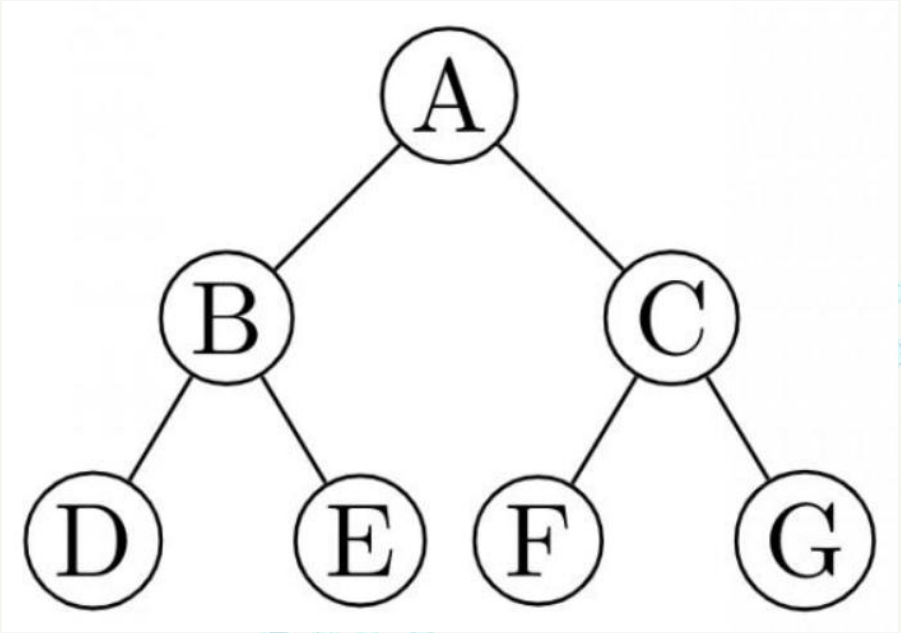
# DFS vs BFS

- DFS is commonly implemented using a stack or recursion.
- Compare this with the queue used in BFS:

| Feature | BFS | DFS |
|---|---|---|
| Data structure | Queue | Stack |
| Principle | FIFO | LIFO |
| Exploration | Level by level | Deepest branch first |
| Backtracking | Not the main strategy | Essential |
| Shortest path | Yes, with equal step costs | Not guaranteed |

Rasheed

# DFS traversal example




Suppose goal is G:

Traversal order: A → B → D → E → C → F → G

Solution path: A → C → G

Prepared By: Mamun Rasheed

# The Backtracking Process

- When it hits a dead end or a node with no unvisited neighbors, it backtracks to the previous node and explores other unvisited paths
- DFS returns to an earlier node and checks whether another branch remains
- In our example:
  - A → B → D
  - D has no children, so return to B.
  - B has another child E, so explore E.
  - E has no children, so return to B and then A.
  - A has another child C, so continue through C → F → G.
- This return to an earlier decision point is called backtracking.

Prepared By: Ehtisham Rasheed

# DFS Pseudocode

```
DFS(start, goal):

    create an empty stack
    create an empty visited set
    push start onto the stack

    while stack is not empty:

        current = pop the top node

        if current is not visited:

            mark current as visited

            if current is the goal:
                return success

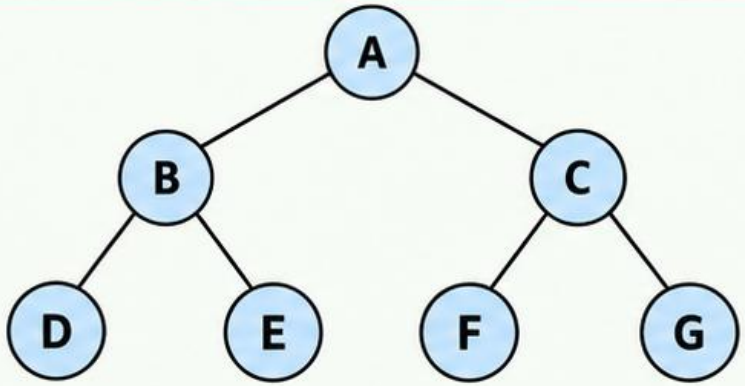
            for each unvisited neighbor of current:
                push neighbor onto the stack

    return failure
```

# Depth-First Search (DFS) Step by Step

(Mark nodes visited when pushing them onto the stack)

## Example Graph




## Adjacency List

| Node | Neighbors (Left to Right Order) |
|---|---|
| A | B, C |
| B | D, E |
| C | F, G |
| D | – |
| E | – |
| F | – |
| G | – |

## DFS Rules (Mark Visited on Push)

1. Initialize stack with the start node and mark it visited.
2. While stack is not empty:
   3. Pop the top node from the stack.
   4. Process the node (add to traversal order).
   5. For each unvisited neighbor (left to right), push it onto the stack and mark it visited immediately.
6. Repeat until the stack becomes empty.

## Step 0: Initialize

**Action**
Push start node **A** and mark it visited.

**Current Node**
–

**Stack** (bottom → top)
[ A ]

**Visited Set**
{ A }

## Step 1: Pop A and push its neighbors

**Action**
Pop **A**.
Push **C, then B** (mark both visited).

**Current Node**
A

**Stack** (bottom → top)
[ C , B ]

**Visited Set**
{ A, B, C }

## Step 2: Pop B and push its neighbors

**Action**
Pop **B**.
Push **E, then D** (mark both visited).

**Current Node**
B

**Stack** (bottom → top)
[ C , E , D ]

**Visited Set**
{ A, B, C, D, E }

## Step 3: Pop D

**Action**
Pop **D**.
No unvisited neighbors to push.

**Current Node**
D

**Stack** (bottom → top)
[ C , E ]

**Visited Set**
{ A, B, C, D, E }

## Step 4: Pop E

**Action**
Pop **E**.
No unvisited neighbors to push.

**Current Node**
E

**Stack** (bottom → top)
[ C ]

**Visited Set**
{ A, B, C, D, E }

## Step 5: Pop C and push its neighbors

**Action**
Pop **C**.
Push **G, then F** (mark both visited).

**Current Node**
C

**Stack** (bottom → top)
[ G , F ]

**Visited Set**
{ A, B, C, D, E, F, G }

## Step 6: Pop F

**Action**
Pop **F**.
No unvisited neighbors to push.

**Current Node**
F

**Stack** (bottom → top)
[ G ]

**Visited Set**
{ A, B, C, D, E, F, G }

## Step 7: Pop G

**Action**
Pop **G**.
No unvisited neighbors to push.
Stack is now empty.
**DFS is complete.**

**Stack** (bottom → top)
[ G ]

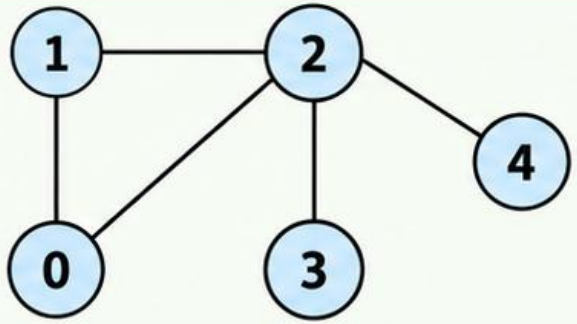
**Visited Set**
{ A, B, C, D, E, F, G }

## Final DFS Traversal Order: A → B → D → E → C → F → G

# Depth-First Search (DFS) Step by Step (Graph 2)

(Mark nodes visited when pushing them onto the stack)

## Graph



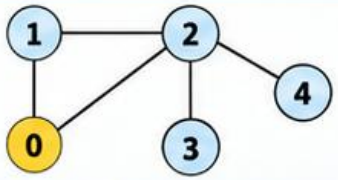

## Adjacency List (Neighbors in Ascending Order)

| Node | Neighbors |
|---|---|
| 0 | 1, 2 |
| 1 | 0, 2 |
| 2 | 0, 1, 3, 4 |
| 3 | 2 |
| 4 | 2 |

## DFS Rules (Mark Visited on Push)

1. Initialize stack with the start node and mark it visited.
2. While stack is not empty:
   3. Pop the top node from the stack.
   4. Process the node (add to traversal order).
   5. For each unvisited neighbor (in ascending order), push it onto the stack and mark it visited immediately.
6. Repeat until the stack becomes empty.

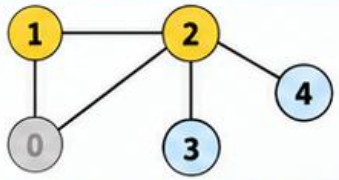
## Step 0: Initialize




| Stack (bottom → top) | Visited Set |
|---|---|
| [ 0 ] | { 0 } |

**Action:** Push start node 0 and mark it visited.

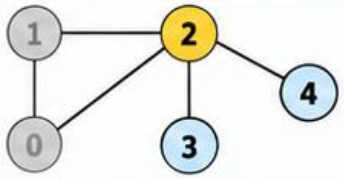
## Step 1: Pop 0 and push its neighbors




| Stack (bottom → top) | Visited Set |
|---|---|
| [ 2 , 1 ] | { 0, 1, 2 } |

**Action:** Pop 0. Push 2, then 1 (mark both visited).

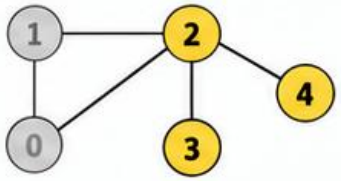
## Step 2: Pop 1




| Stack (bottom → top) | Visited Set |
|---|---|
| [ 2 ] | { 0, 1, 2 } |

**Action:** Pop 1. Neighbors 0 and 2 are already visited. No push.

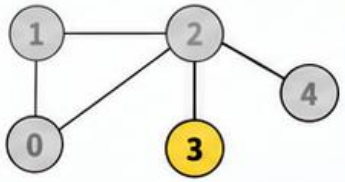
## Step 3: Pop 2 and push its neighbors




| Stack (bottom → top) | Visited Set |
|---|---|
| [ 4 , 3 ] | { 0, 1, 2, 3, 4 } |

**Action:** Pop 2. Push 4, then 3 (mark both visited).

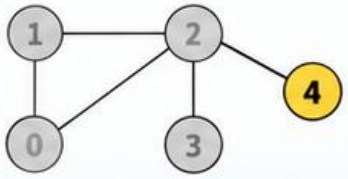
## Step 4: Pop 3




| Stack (bottom → top) | Visited Set |
|---|---|
| [ 4 ] | { 0, 1, 2, 3, 4 } |

**Action:** Pop 3. Neighbor 2 is already visited. No push.

## Step 5: Pop 4




| Stack (bottom → top) | Visited Set |
|---|---|
| [ ] | { 0, 1, 2, 3, 4 } |

**Action:** Pop 4. Neighbor 2 is already visited. No push.
Stack is now empty. **DFS is complete.**

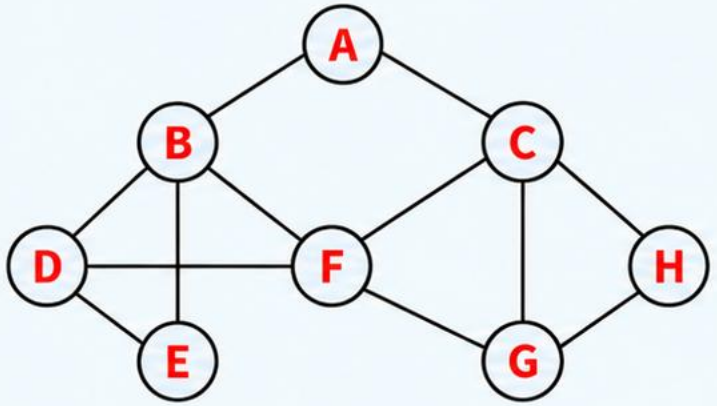
## Final DFS Traversal Order: 0 → 1 → 2 → 3 → 4

# Trace the stack and visited set / Summary Table

| Step | Node popped | New nodes pushed | Stack after step | Visited set |
|---|---|---|---|---|
| 0 | — | `0` (start) | `[0]` | `{0}` |
| 1 | `0` | `2, 1` | `[2, 1]` | `{0, 1, 2}` |
| 2 | `1` | None | `[2]` | `{0, 1, 2}` |
| 3 | `2` | `4, 3` | `[4, 3]` | `{0, 1, 2, 3, 4}` |
| 4 | `3` | None | `[4]` | `{0, 1, 2, 3, 4}` |
| 5 | `4` | None | `[]` | `{0, 1, 2, 3, 4}` |

# Depth-First Search (DFS) – Practice Problems (With Answers)

## 1 Graph 1




### Adjacency List (Graph 1)

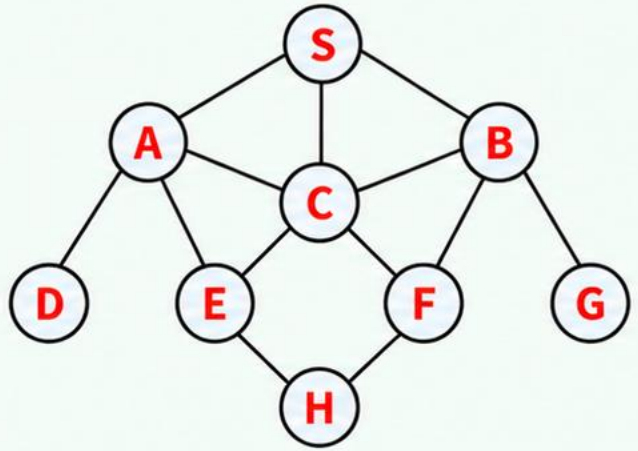
| Node | Neighbors (in order) |
|---|---|
| A | [B, C] |
| B | [A, D, E, F] |
| C | [A, F, G, H] |
| D | [B, E, F] |
| E | [B, D] |
| F | [B, C, D, G] |
| G | [C, F, H] |
| H | [C, G] |

**Start Node:** A **Goal Node:** H

**DFS Traversal Order:** A → B → D → E → F → C → G → H

**Solution Path (A to H):** A → C → H

## 2 Graph 2




### Adjacency List (Graph 2)

| Node | Neighbors (in order) |
|---|---|
| S | [A, B, C] |
| A | [S, C, D, E] |
| B | [S, C, F, G] |
| C | [S, A, B, E, F] |
| D | [A] |
| E | [A, C, H] |
| F | [B, C, H] |
| G | [B] |
| H | [E, F] |

**Start Node:** S **Goal Node:** H

**DFS Traversal Order:** S → A → D → E → H → F → C → B → G

**Solution Path (S to H):** S → A → E → H

# DFS Properties

- Is DFS Complete?
  - Completeness means: Is the algorithm guaranteed to find a solution if one exists?
  - Yes, DFS is complete on finite graph
- Is DFS Optimal?
  - Optimality means finding the solution with the lowest path cost
  - DFS does **not** guarantee the shortest path between nodes
- Time and Space Complexity
  - Let **b** = branching factor and **m** = maximum depth of the search tree
- DFS has the following commonly taught worst-case complexities:
  - Time — Tree = $O(b^m)$ — Graph = $O(V + E)$
  - Space — Tree = $O(b^m)$ — Graph = $O(V)$

Prepared By: Ehtisham Rasheed

# Advantages and Disadvantages

- Advantages
  - DFS generally uses less memory than BFS when the branching factor is large and the search is deep
  - It can be implemented using a stack or recursion
  - DFS can find a solution quickly if the goal lies along a branch it explores early
- Disadvantages
  - DFS may find a long solution before discovering a shorter one
  - In an infinite search tree, DFS may never backtrack to explore other branches
  - Changing the order in which neighbors are explored can change the traversal and the first solution found

Prepared By: Ehtisham Rasheed

# Practical AI Applications

- Maze Solving
  - Explore a corridor until reaching a dead end, then backtrack and try another corridor. DFS can find a route, but not necessarily the shortest one
- Dependency Analysis
  - Explore dependencies between software modules, packages, or tasks. DFS is also used in algorithms for topological sorting of directed acyclic graphs
- Network and Connectivity Analysis
  - Determine which nodes are reachable, discover connected components, and analyze network structure
- Puzzle solving and Backtracking
  - Explore possible choices in puzzles such as Sudoku, maze puzzles, and constraint-satisfaction problems. DFS can backtrack when a choice leads to a dead end

Prepared By: Ehtisham Rasheed